Blender ↔ ROS 2 Bridge and Sensor Simulation: Research Findings

2026-10-18. Measurements are from this machine: Blender 5.2.2 snap (Python 3.13.13), ROS 2 Humble (Python 3.10), RTX 4060 8 GB, i7-13620H.

Existing bridges: nothing usable

Project	Status
geni-lab/ros_blender_bridge	ROS 1, Blender 2.71, dead since 2014
ahmad-aljabali/ROS-Blender-Addon	ROS 1 rospy inside Blender, 2020, no license
b-robotized/sr_ros2_blender_tools	ROS 2 camera only, 2 commits (2023)
LinkForge	URDF export only, no runtime

All of them assume rospy/rclpy can be imported inside Blender. **That is impossible here:** Humble's rclpy is cp310, Blender 5.2 is cp313, and the `bpy` wheels are cp311 (4.2–5.0) or cp313 (5.1+).

Transport options

Option	Verdict
Offline MCAP via `rosbags` (pure Python, supports 3.13, MCAP writer)	**Use for sensor data.** Deterministic, sim time. Tested: a Blender-written CustomMsg bag reads fine in Humble.
Zenoh (`eclipse-zenoh` abi3 wheel on 3.13 + `zenoh-bridge-ros2dds`)	Good live option, but an extra system dependency
ZMQ/TCP to a separate rclpy process	Simple; **we use a dependency-free TCP variant for the live viewer bridge**
roslibpy + rosbridge	JSON+base64, about 5–20 MB/s. Too slow for clouds and images
bpy pip module / ROS rebuilt for 3.13	No matching interpreter, or too much work

Measured performance

Task	Throughput
`BVHTree.ray_cast` (Python loop in Blender)	115k rays/s
`Scene.ray_cast`	42k rays/s
embreex inside Blender (`pip --target`)	541k rays/s
Open3D `RaycastingScene` (Python 3.10,	**3.5M rays/s** (Airy needs ~0.86–1.7M, Mid-360 0.2M)

Task	Throughput
Embree, multithreaded)	
EEVEE 1080p, one eye	0.29 s per frame (36 s first-frame shader compile). Needs NVIDIA EGL: `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json`
Cycles OptiX 1080p, 32 spp + denoise	~2 s per frame

Stereo: `render.use_multiview`, PARALLEL convergence, `interocular_distance=0.12`.

Scan patterns

- **Mid-360:** `Livox-SDK/livox_laser_simulation/scan_mode/mid360.csv` (MIT).
  - Header `Time/s,Azimuth/deg,Zenith/deg`, 800k rows. Zenith 37.84–97.21° (elevation +52.2… −7.2°).
  - Step through rows at 200 kHz (offset 5 µs). `line = i % 4`, `tag = 0x10`.
- **Airy:** 96 channels. Vertical angles come from per-unit DIFOP calibration; the manual says 0–90° at ~0.947° spacing, so assume uniform.
  - Firing offsets are in rs_driver `decoder_RSAIRY.hpp` (block 111.08 µs).

IMU from the trajectory

- Quintic position spline plus scipy `RotationSpline`.
- Specific force `f_b = R_wb^T (a_w − g_w)`.
- Lever arm: `a + α×r + ω×(ω×r)`.
- Noise: white noise (σ_c·√rate) plus bias random walk.
- Livox and Airy raw IMUs report in **g**.

Handheld motion

Walk at ~1.4 m/s. Add vertical bob of ±2–3 cm at 1.8–2 Hz, yaw sway, hand tremor (0.2–1°, 4–12 Hz), occasional pans, and a static start of a few seconds.